ETE7-CNUM1 – Codage de Canal

TP de codage canal

A. Valade et R. Tajan

1 Objectifs et évaluation

L'objectif de ce TP de codage canal est de simuler à l'aide du logiciel Matlab des chaînes de communications numériques codées afin d'évaluer notamment leurs performances en termes de probabilité d'erreur binaire, en termes de rendement, mais aussi en termes de débit.

Les TP se font en **binôme ou monôme** et l'évaluation porte sur une note de rapport et une note de travail continu (en séances).
Concernant le rapport, il ne doit pas excéder **15 pages**, vous devez fournir un document scientifique et technique, qui doit présenter votre travail, vos choix techniques et dans lequel **tous les résultats obtenus doivent être interprétés et commentés**. Les codes Matlab doivent également être transmis à votre enseignant. Ils doivent pouvoir être compris rapidement. Cela passe par l'utilisation de commentaires. Les commentaires doivent permettre de répondre au moins à la question : que fait la ligne de code ? Une attention particulière doit être portée à la lisibilité du programme. Les rapports et les codes doivent être rendus par l'intermédiaire de la plateforme Thor **avant la fin de la dernière séance pour votre groupe**.*

Vous trouverez les fichiers initiaux pour réaliser ce TP à l'adresse suivante
`https://github.com/rtajan/TS226`.

2 Codes convolutifs

Dans ce TP, vous allez vous intéresser au cas des communications numériques codées à l'aide d'un code convolutif. Vous devrez implémenter, par vous-même, les fonctions d'encodage et de décodage des codes convolutifs. L'architecture de communication à considérer est présentée sur la Figure 1.

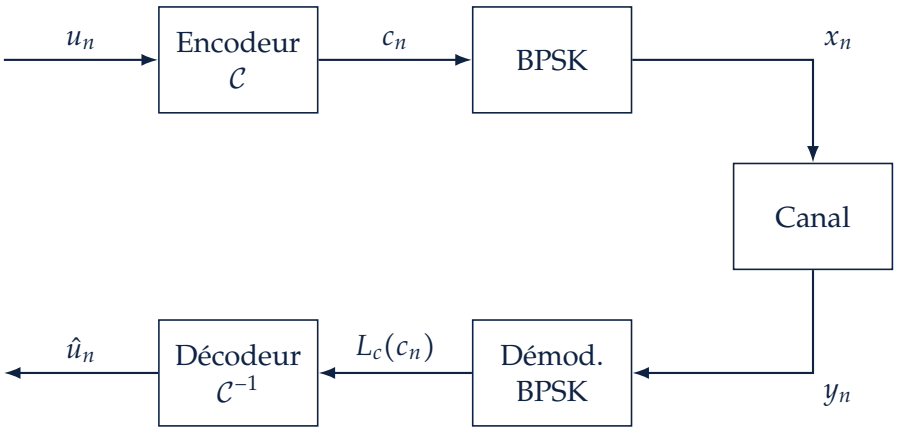


FIGURE 1 – Chaîne de transmission codée

*. Tout retard peut faire l'objet d'un refus de consulter votre rapport, l'heure de dépôt sur Thor faisant foi.

2.1 Treillis

La communication toolbox de Matlab offre une structure de donnée permettant de représenter simplement les diagrammes d'états des codes convolutifs. Ces structures sont construites en utilisant la fonction `poly2trellis` de Matlab. Cette fonction prend comme premier argument la longueur de contrainte $\nu = m + 1$ et le polynôme en notation octale sous forme de vecteur. Par exemple, la commande `poly2trellis(3,[7,5],7)` créant l'encodeur $\left(1, \frac{5}{7}\right)_8$ renvoie la structure suivante :

- `numInputSymbols` : 2 (nombre de symboles possibles en entrée, 2^{n_b})
- `numOutputSymbols` : 4 (nombre de symboles possibles en sortie, 2^{n_s})
- `numStates` : 4 (nombre d'états possibles en sortie, 2^m)
- `nextStates` : tableau représentant la fonction suivante $s_{n+1} = f(s_n, u_n)$

État \ Entrée	0	1
0	0	2
1	2	0
2	3	1
3	1	3

TABLE 1 – Tableau représentant les états suivants à partir des états courants (lignes) et des symboles en entrée (colonnes).

- `outputs` : tableau représentant la fonction suivante $c_n = g(s_n, u_n)$

État \ Entrée	0	1
0	0	2
1	0	2
2	1	3
3	1	3

TABLE 2 – Tableau représentant les sorties à partir des états courants (lignes) et des symboles en entrée (colonnes).

2.2 Encodage (Matlab)

Écrire une fonction `cc_encode` ayant le prototype suivant :

```
function c = cc_encode(u, trellis).
```

Cette fonction possède 2 arguments en entrée :

- **u** : un vecteur de K symboles d'entrées.
- `trellis` : une structure représentant le treillis.

Cette fonction renvoie 1 sortie :

- **c** : le mot de code.

L'état initial sera systématiquement l'état 0 et le treillis devra être fermé. L étant le nombre de sections fermeture incluse et K le nombre de symboles du message. Comme le treillis est fermé, **c** devra être de taille $n_s L$.

2.3 Décodage de Viterbi (Matlab)

Écrire une fonction `viterbi_decode` ayant le prototype suivant :

```
function u = viterbi_decode(y, trellis).
```

Cette fonction renvoie **u** un vecteur de message de taille K. **y** est un vecteur de $n_s L$ observations du canal.

Astuces :

1. le choix de l'implémentation de l'algorithme de Viterbi est laissé libre. Cependant, il est souvent pratique de stocker les métriques de branches dans un tableau de taille $2^m \times (L+1)$.
2. vous pouvez utiliser l'outil `bertool` de Matlab pour tester votre décodeur.

2.4 Étude de l'impact de la mémoire du code (rapport)

Utiliser les codes développés précédemment pour réaliser une étude comparative de l'impact de la mémoire des codes convolutifs leurs performances. Pour cela, pour $K = 1024$, vous simulerez les performances de décodage des codes convolutifs suivants :

- $(2, 3)_8$
- $(5, 7)_8$
- $(13, 15)_8$
- $(133, 171)_8$

Sur un même graphique, tracer les taux d'erreur binaire (TEB) en fonction de E_b/N_0 [†] pour chacun des codes de la question précédente. Sur ce graphique, tracer aussi la courbe de la probabilité d'erreur binaire dans un scénario non codé.

Pour chaque stratégie, relever le **gain de codage** sur les courbes obtenues. Le gain de codage est la différence en dB entre la valeur E_b/N_0 pour laquelle une stratégie non codée passe sous $P_b = 10^{-5}$ et la valeur E_b/N_0 pour laquelle une stratégie codée passe sous $P_b = 10^{-5}$.

Une autre caractéristique intéressante est le débit de sortie du récepteur. Ce débit est normalement déjà évalué par le script fourni. Pour chaque code, vous relèverez le débit lors du calcul de la dernière valeur de E_b/N_0. Dans votre rapport, après avoir présenté les courbes de résultat, faites un tableau de synthèse récapitulatif explicitant pour chaque code :

- son rendement **lorsque le treillis est fermé**
- sa mémoire et leur nombre d'états
- sa distance minimale (obtenue à l'aide de la fonction `distspec`)
- son gain de codage
- le débit de décodage

Réaliser votre étude comparative à l'aide du tracé du TEB en fonction de E_b/N_0 et du tableau récapitulatif.

2.5 Encodage récursif / encodage non récursif (rapport)

Tracer sur un même graphique les performances pour le décodage de Viterbi des codes convolutifs pour $K = 1024$ avec les encodeurs suivants :

†. Les valeurs de E_b/N_0 iront de −2dB à 10dB par pas de 0.5dB. Le pas peut être ajusté en fonction des stratégies de codage. Il n'est pas nécessaire de simuler les points de E_b/N_0 ayant un TEB inférieur à 3×10^{-6}.

- $(5, 7)_8$
- $\left(1, \frac{5}{7}\right)_8$
- $(13, 15)_8$
- $\left(1, \frac{13}{15}\right)_8$

Le décodage de Viterbi sera considéré treillis fermé. Dans votre rapport, présentez les résultats obtenus, analysez les courbes et apportez une démonstration des résultats obtenus.

3 Pour aller plus loin...

3.1 Codes courts

Au cours de ce TP, vous avez travaillé avec des messages de longueur $K = 1024$ bits d'information. Dans la réalité, la longueur de vos messages va dépendre de votre standard. Certains standards, comme le *digital video broadcasting – satellite – second generation* (DVB-S2) vont encoder des messages de longueurs $K = 16\,200$ bits pour les plus courts et $K = 64\,800$ bits pour les plus longs. A contrario, d'autres standards peuvent exiger des trames très courtes, dont la longueur peut aller de $K = 32$ bits à $K = 128$.

Nous allons nous intéresser ici à ce second cas. Reprenez les codes que vous avez testés précédemment et testez-les pour une longueur de messages de $K = 64$, avec un trellis fermé. En comparant avec vos résultats précédents, qu'observez-vous ?

Présentez ces nouveaux résultats dans votre rapport. Mettez en parallèle le rendement et la distance minimale pour les deux valeurs de K. Que remarquez-vous ? Comment cela a-t-il impacté le décodage ?

Bonus : Quels seraient les avantages et inconvénients à travailler avec des messages très courts ? Dans quel genre de standards pensez-vous que ces codes s'intègrent ?

3.2 Codes expurgés

En restant dans le paradigme des codes courts, on vous propose d'explorer une autre méthode de construction de code basée sur l'expurgation.

Pour rappel, **expurger** un code consiste à éliminer ses mots de code de poids faible. Dans un code linéaire en bloc cela se fait en supprimant certaines lignes de la matrice génératrice **G**. Cette opération change la dimension des messages et fait en sorte que ceux encodant les mots de code de poids faible ne soient plus possibles.

Pour les codes convolutifs, on peut procéder différemment. Une méthode consiste à utiliser des **fonctions d'expurgation linéaire** (ELF). Celles-ci fonctionnent comme des encodeurs à contrôle de redondance cyclique (CRC) et créent un code dit **concaténé** dont le spectre de distance est *expurgé* par rapport au code convolutif initial. Ce code conaténé est présenté dans la Figure 2

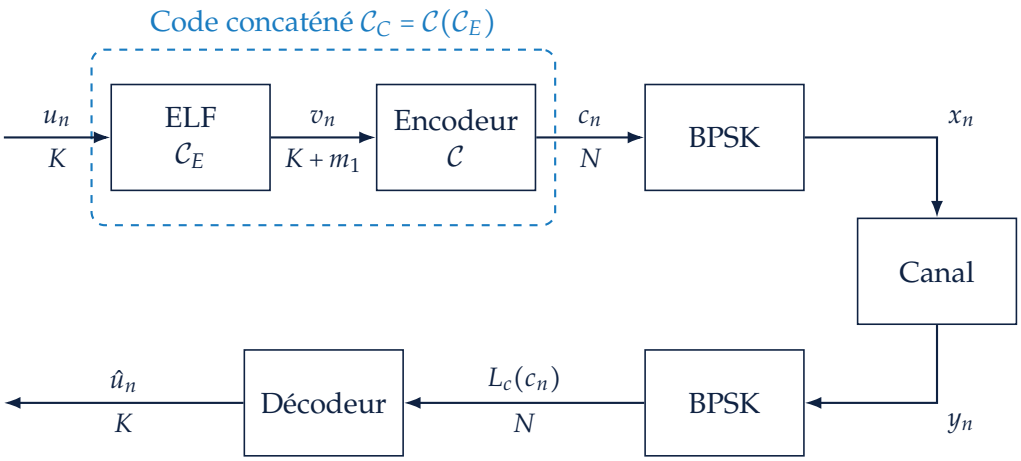


FIGURE 2 – Illustration de la concaténation ELF/Convolutif dans la chaîne de communications du TP

Pour $K = 64$ et à l'aide de `distspec`, comparez les quatre premiers éléments du spectre de distance du code concaténé obtenu à l'aide du CRC 0x1f et du code convolutif $(13, 17)_8$. Comparez-le au spectre de distance du code $(133, 171)_8$.

Indice : Pour obtenir le spectre de distance du spectre concaténé, faites la multiplication du polynôme du CRC à ceux du code convolutif.

On vous donne la formule du FER théorique par décodage ML.

$$P_e = \sum_{w=d_{\min}}^{+\infty} A_w Q\left(\sqrt{2wR\frac{E_b}{N_0}}\right) \tag{1}$$

En utilisant (1) sur les spectres que vous avez obtenus, comparez les deux codes entre eux. Lequel achève le FER théorique le plus bas ? Pour quel SNR ? Quel gain de codage entre les deux codes ?

Dans votre rapport, poussez l'analyse : ces deux codes ont-ils le même rendement ? Quelle est la différence ? Comment l'interprétez-vous ?

Si vous avez le temps... à partir de votre cours et de la structure du code concaténé, proposez une méthode de décodage pour ce code. Justifiez votre choix.

Si vous avez vraiment le temps... écrivez une fonction `crc_encode` pour réaliser l'encodage de votre code externe. Celle-ci doit prendre comme arguments **u** votre vecteur de message et **g** un vecteur de coefficients binaires décrivant le polynôme de votre code externe. Testez votre méthode de décodage. Obtenez-vous les performances prédites par (1) ?

4 Contacts

– Romain Tajan – romain.tajan@ims-bordeaux.fr

– Alexandre Valade – alexandre.valade@ims-bordeaux.fr